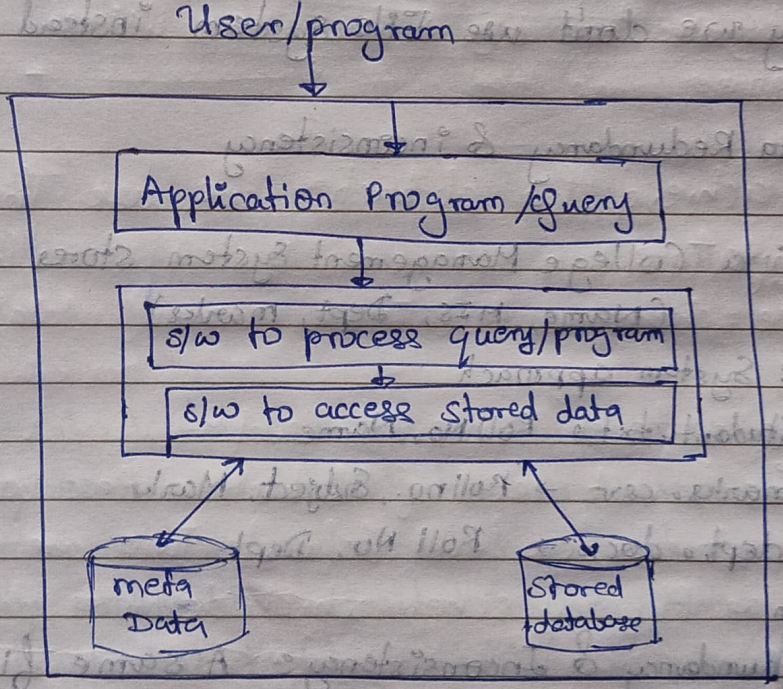
DBMS

Database :- collection of related data.
↳ represent part of reality / implicit meaning.
→ It is organized meaningful, not random data.
→ It is created for specific use & spefic user.

Database Management System :- DBMS
↳ a collection of interrelated data & set of program to manage the data.

Ex Student (MIS , Name)
Integer, string 80 → type of data of property.
Student → table Name
MIS, Name → columns / properties / attribute



Example :- Create a Database for student details :-
→ student :- (Name, MIS, class, major)
→ Course (Cname, Cno, credit, Dept)
→ section (section ID, CNo., semester, year, In-charge)
→ Student Report (MIS, Grade, subject)

Terms :-

- Users :- There are different types of users based on what function they can perform. There are 4 types
- View :- How we can visualize database in different ~~view~~ ways (3 types) physical level, logical level, view level
- Model :- A data model defines how data is organised, structured & related.
- key : attribute or group of attributes which can uniquely identify the records

Why we don't use File system instead of DBMS

(1) ~~Data Redundancy~~ & inconsistency

Ex :- a College Management System stores student data (Name, MIS, Dept, marks)

File System approach

Student.txt : Roll No, Name

marks.csv → Roll no., Subject, Marks

dept.docx → Roll No, Dept

Problem

1) Redundancy & Inconsistency :- A same field may repeat in multiple files like Roll No & Name. Suppose a field is updated in one file but not others, then it will cause inconsistency

2) Difficulty in Accessing Data :- Custom programs needed for each query/report. No standard querying language

Three Level Architecture :-

① ~~Schema/ Interface~~ Instance

Ex. Student (~~MIS NO~~, Name, FirstName) → schema
faculty (. . . . .)
course (. . . . .)

the data row in the table is called "instance

---

3. Data Isolation :- Data scattered across files with different formats makes it hard to integrate & retrieve meaningful information.

4. Integrity Problems :- Difficulty to enforce constraints (eg. roll number must be unique, marks >=0), leading to invalid or incorrect data

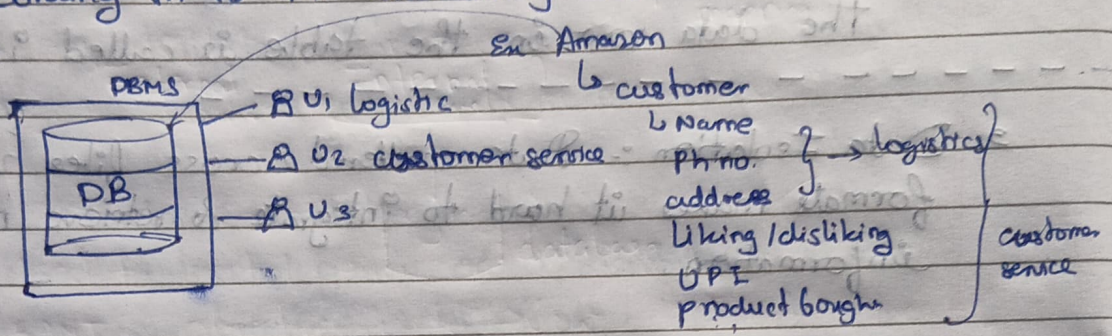
5. Atomicity Problems :- Transactions (series of operations) may fail midway with no way to ensure "all or nothing" execution

6. Concurrent-Access Anomalies :- Multiple users accessing/ updating same data → data loss or corruption.

7. Security Problems :- Limited or no access control, difficult to restrict data visibility. Ex any user opening marks.csv can see all student's marks (no role-based access).

Abstraction :- system is simplified for user, we only give the user info about system enough for operating not the handwiring & mechanism.

The role of DBMS is to provide an abstract view according to the accessing user



The purpose is to make the view according to the user.
Ex logistic has only Name, Ph no, address
Customer service has ~~all~~ + product bought + meta

To achieve this abstraction we use Three Scheme architecture

⇒ Three schema architecture (view of data)
↳ Design

(1) Physical level / Internal level :-

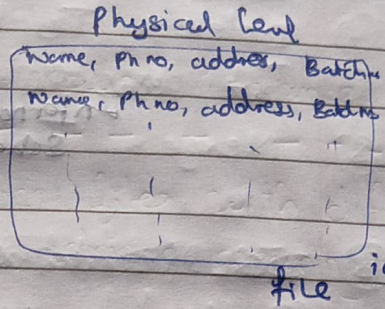
⇒ The lowest level of abstraction describes how data are stored. low level data structures are used
It has Physical schema which describes physical storage structure of DB ↳ blueprint.
Talks about storage allocation (N-ary tree etc), Data compression & encryptic etc.
Goal :- we must define algorithms that allows efficient access to data.
Focus :- Physical storage details like data structures, file organization, indexes, hashing, pointers, B+ trees
⇒ Hardware & storage oriented view

2. Logical level / conceptual level :-

i) The conceptual schema describes the design of db at conceptual level, describes what data are stored in DB & what relationships exist among those data.

ii) User at logical level need not to be aware about physical level structure.

Ex Student → Name
→ Ph No.
→ address
→ Batch no.



Student

	Name	Ph no	address	Batch
①	XYZ	8...	street 1	2019
②	Lekshi	9...	street 2	2022

→ logical → can take a idea of what data is

Goal :- make data easy to use

→ Describes what data is stored & relationships among them. here you only see the table structure not how it's stored on disc

3. View level :- Highest level of abstraction aims to simplify user's interaction with system by providing different view to different end user.

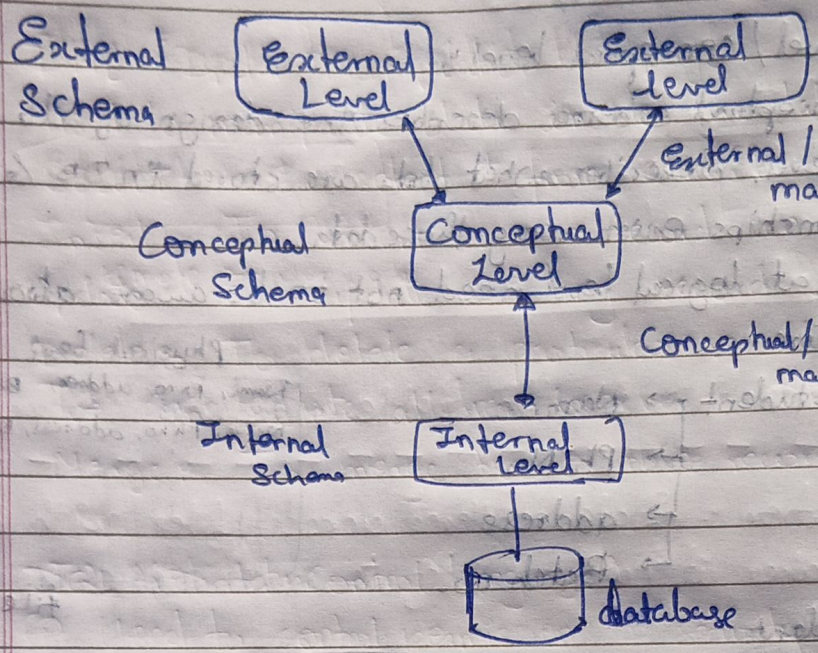
Ex logistic view → | Name | Ph. no | address |

Customer service view → | Name | Ph. no | address | Products buyed |

Each user/app doesn't need or get entire DB, only a relevant portion.

at external level, a db contains several schemas → subschemas which are used to describe the different view of db.

→ at view also provide a security mechanism to prevent users from accessing certain parts of DB



Instance of DB

Student →

	Name	ph no	address	Batch
①	xyz	xx	xx	xx
②	abc	xx	xx	xx

at 12:00 AM 04-06-20xx → 2 rows, 2 data points

The collection of information stored in the DB at a particular moment is called an instance of DB

The overal design of DB is called DB schema

Logical schema being the most important

DB schema (logical schema)

① attributes of table

② Consistency constraints (Primary key) ← not null

③ relationship

Student table ⟶ course table

- Physical data independance :- physical schema change should not affect logical schema / application programs

PAGE NO.
DATE / /

Database Users :-

1. Naive Users
2. Application programmers :-
3. Sophisticated users :-
4. specialized user

Data Modal :- A way to describe how data is represented, stored, & manipulated in a database.
- provides concepts, rules & notations to design & understand a database Ex (ER model, db relations, semantics, consistency constraints)

Database Languages :- used to define, manipulate, control & query data in a database, they are mainly part of SQL, but conceptually divided into types :-

1. Data Definition Language (DDL) :-
   Desiging structure / schema of ~~language~~ the database

Commands

1. CREATE :- Creating the structure
2. ALTER :- Adding an extra column (add / remove / modify)
3. DROP :- It actually drop objects (deletes a table or db)
4. TRUNCATE :- used to reinitialize the table or delete all rows from a table but keeps structure
5. COMMENT :- to add some info in our data dictionary, or to save some type of constraints
6. RENAME :- for renaming the column in a table

PAGE No.
DATE / /

② DML (Data Manipulation Language
Used to manipulate data inside the table
① SELECT :- Eg select * from students
② Insert :- To insert a new record in database
③ Update :- To update existing record in database
④ Delete :- To delete a particular record in database or we can delete all records
⑤ Merge :- To merge two or more records

③ DCL (Data Control language)
Used to control permissions & security
① GRANT :- objective is to allow different types of permission to user.
② REVOKE :- take permission back or disallow the given ~~perfo~~ permission

④ DQL (Data Query language)
Used to query (retrieve) data.
SELECT :- fetches data from one or more tables

⑤ TCL (Transaction Control language)
Used to manage transaction (group of operation that must succeed or fail as one unit)

① COMMIT :- saves all changes permanently in our database
② SAVEPOINT - ~~Undoes changes since last COMMIT.~~
Creates a checkpoint to roll back partially

③ ROLLBACK :- Undoes changes since last COMMIT

Database Users :-

There are different types of methods to interact with a database & there are different types of users

(1) Naive Users :- Little or no knowledge of DB.
Use GUI forms or applications to access DB.

(2) Application Programmer :- Needs to write code to interact with database.
- Use RAD tools to build apps.

(3) Sophisticated users :- write queries directly (SQL) & has deep knowledge of DB structure

(4) Specialized Users :- Work with complex application (eg AI, scientific research).
Needs deep domain + DB knowledge.

Database Administrator (DBA)

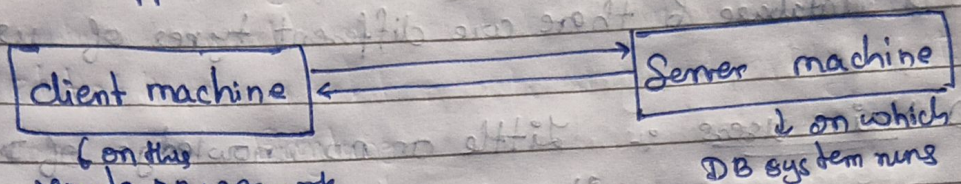
a. A person who has central control of both the data & programs that access those data

DBA works mostly with internal & logical level

functions of DBA

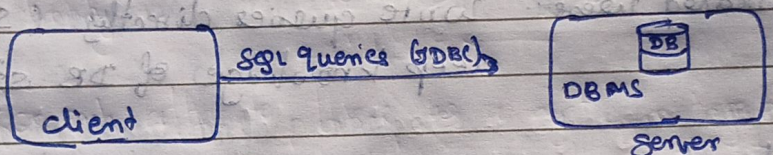
(i) Schema Definition
(ii) Storage structure & access methods
(iii) Schema & physical organization modifications
(iv) Authorization control
b) Routine maintenance
1) periodic backups
2) Security patches
3) Any upgrades

DBMS application architecture



① T1 architecture → client, server & DB are all on same machine

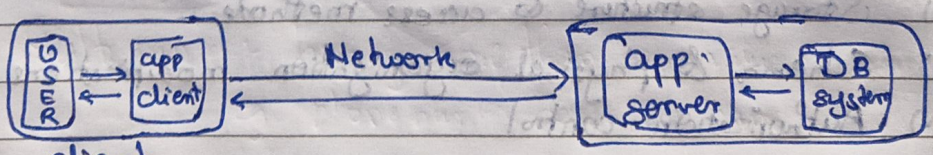
② T2 Architecture



Client machine invokes DB system functionality at server end through query language statements

③ T3 Architecture

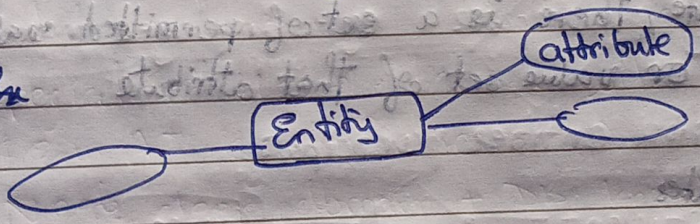
- App is partitioned into 3 logical component
- Client machine is just frontend & doesn't contain any direct DB calls.
- Client machine communicates with app server, & app server communicates with DB system to access data.
- Business logic is implemented in app server.
- T3 architecture are best for WWW applications

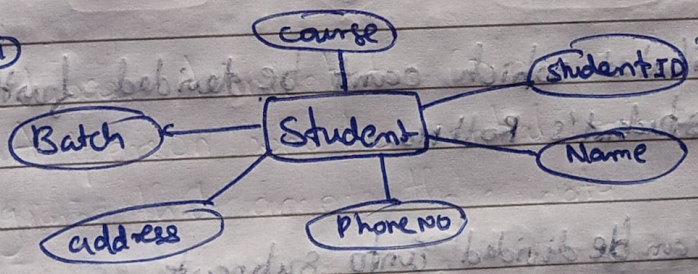


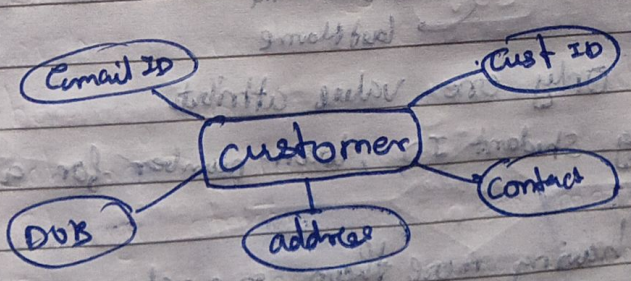
Advantages :- 1) Scalability
2) Data Integrity
↳ app server as a middle layer minimize chances of data corruption.
3) Security :- client can't directly access DB. (more secure)

ER model (Entity Relationship model)
↳ a data model

① It is a high level data model based on perception of real world that consists of a collection of basic objects, called entities & of relationships among these objects

Ex 

Ex ① 

② 

Entity set :- It is a set of entities of the same type that share the same properties or attributes.

Attributes :- They are used to define entity set

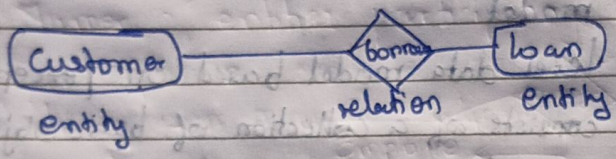
Relationships

Eg Banking

Customer, loan, account

① Customer borrow loan.

② Customer deposit account



* for each attribute, there is a set of permitted values called domain or value set of that attribute

Types of Attributes

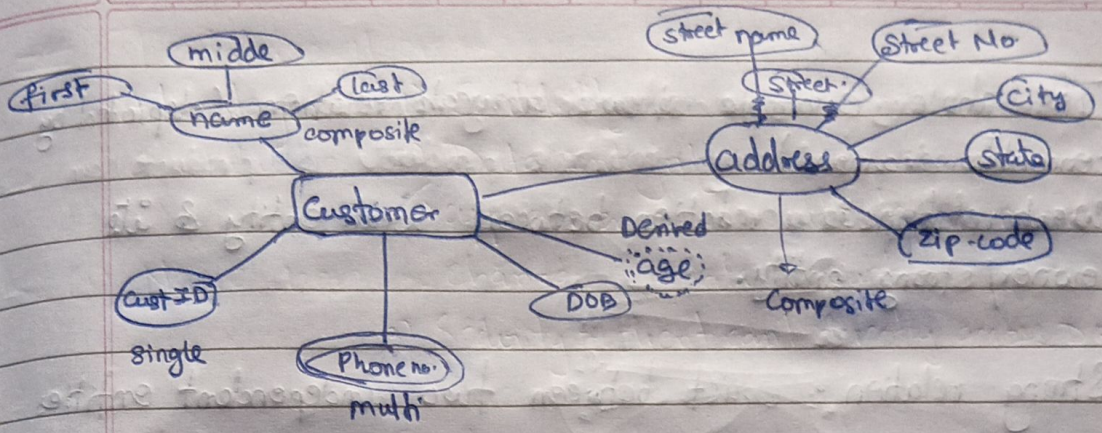
1) Simple :- attributes which cant be devided further
Eg. student's Roll No.

2) Composite :- can be divided into subparts
Eg Name → firstName
→ middleName
→ lastName

3) Single-valued :- only one value attribute
* Eg Student ID, loan-number for a loan

4) Multi-varied :- having more than one value
eg:- phone number, dependent name

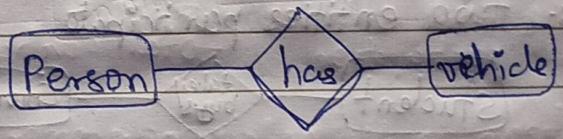
5) Derived :- Eg Age, loan age, membership period.



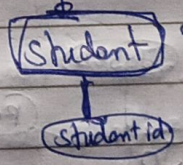
NULL Value :- - when entity does not have a value for it
- It may indicate "not applicable", value doesn't exist
eg. person having no middle name

- It may indicate "unknown". (also indicate missing entry)

Relationships :- association among 2 or more entities

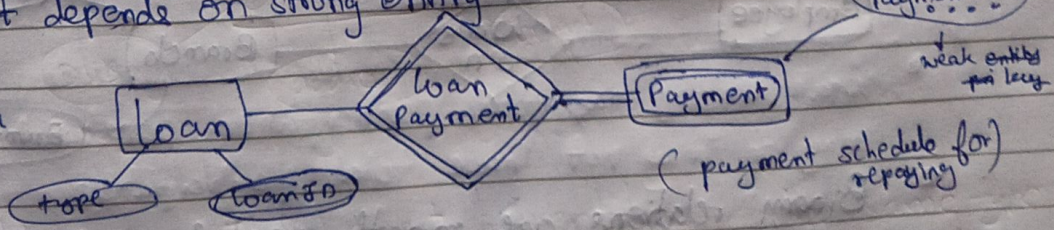


strong entity :- which has an independent existence
(for eg university DB



can be recogized by a primary key

Weak entity :- does not have an independent existence.
It depends on strong entity



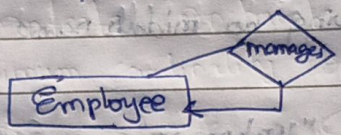
Payment ~~no is wea~~ is also a weak entity as it exist only when there exist a loan.

The above example shows a weak relation (depicted by concentric diamond)
Weak relation is a relation between weak entity & its owner/strong entity.

Strong relation :- exist between two independent entities

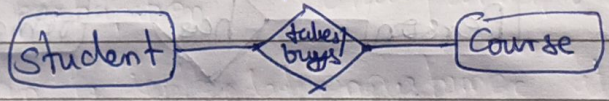
Degree of relationship
→ Number of entities participating in a relationship.

① Unary :-

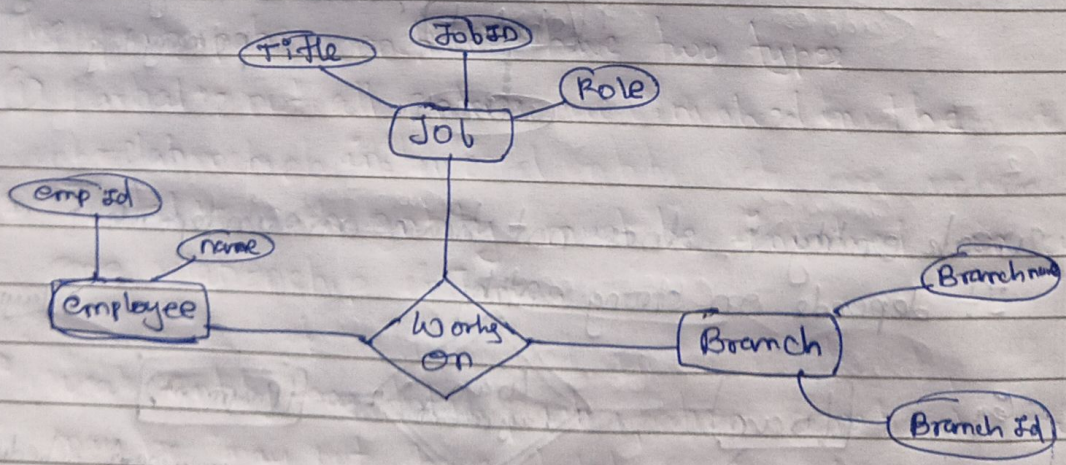


Only one entity participates eg:- Employee manages employee

② Binary :- two entities participate

eg 

③ Ternary :-



* Binary relations are mostly used & most common.

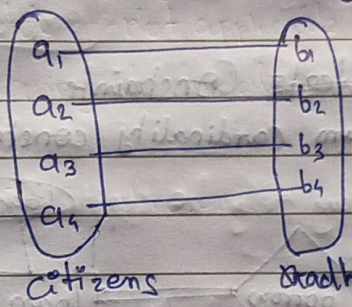
Relationship constraints

① Mapping Cardinality / Cardinality Ratio

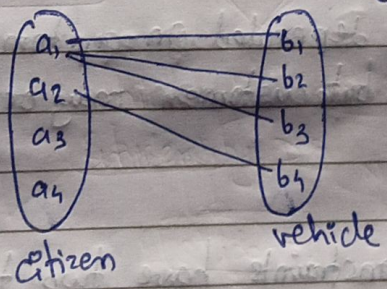
⇒ Number of entities to which another enitity can be associated via a relationship

1:1 Eg: citizen & aadhar card

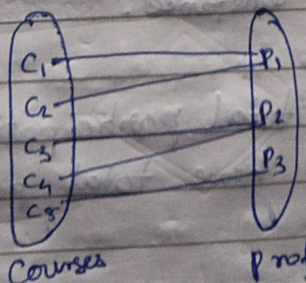
a citizen can have only one aadhar card & vice versa.



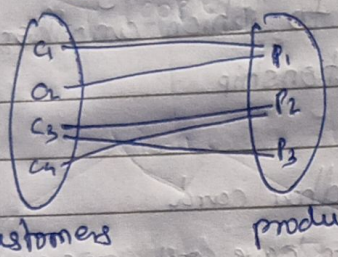
1: Many Ex citizen & vehicle, a citizen can multiple cars but one car can have only one hear



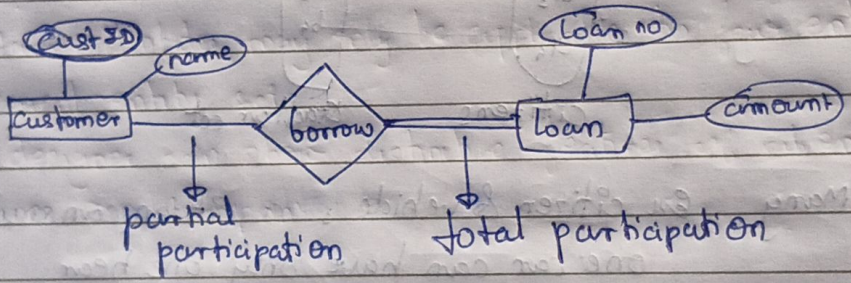
Many:1 Eg. Course taken by professor, one course has one professor, but one prof can have n courses



(3) Many : Many Ex. customer buy product
customer can buy multiple products & products can have multiple buyers



(4) Participation ~~Constrain~~ Constraints
(also known "minimum cardinality constraint")

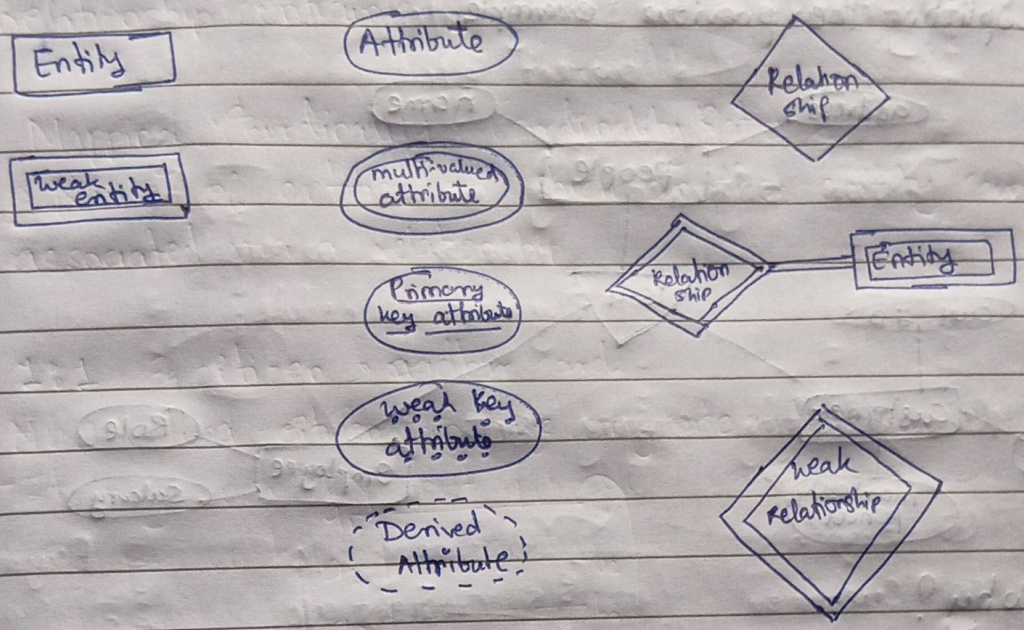


as loan is depended on customer it has total participation, but customer may or may not have a loan

The participation constraints have two types
(1) Partial :- not all entries are involved in the relationship instance
(2) Total :- each entity must be involved in at one relationship instance.

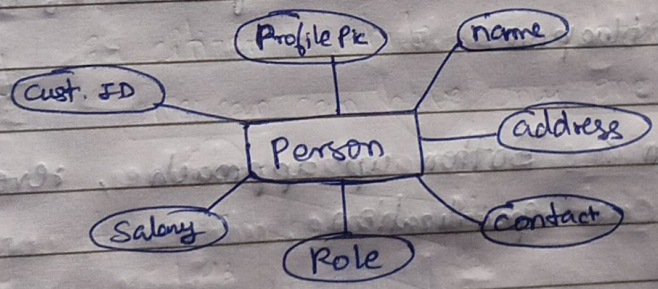
* weak entity has total participation constraint but may or may not have total participation

ER notation



Extended ER features

Ex

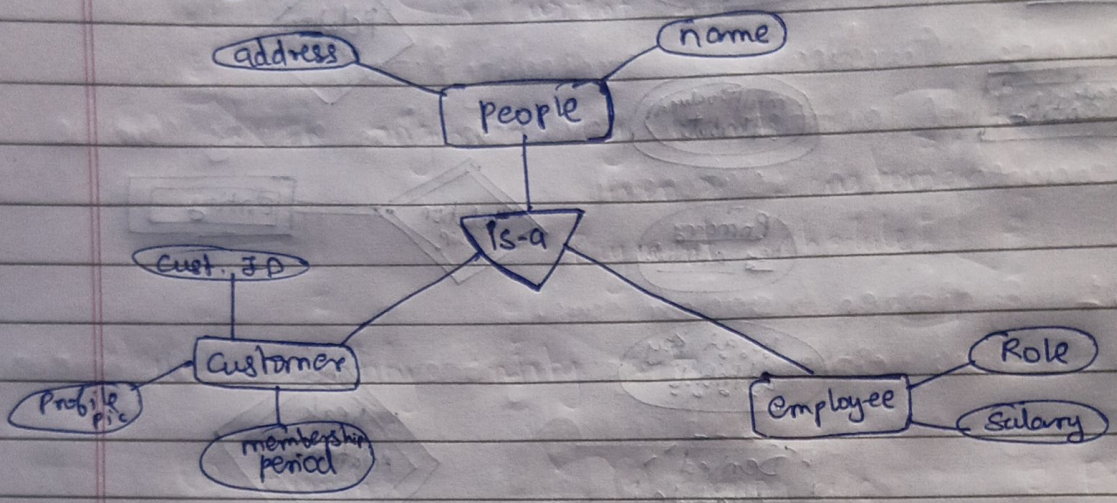


with more complexity the ER becomes redundant

Specialization: splitting up the entity set into further sub entity sets on the basis of their foundah functionality, speciality & features.
Top-down approach

The relation is depicted by ▽ through is-a relationship between superclass & subclass

Splitting the previous example



Why?

- certain attributes may only be applicable to few entities of parent entity set.

→ DB designer can show distinctive features of sub entities
→ refines DB blueprint

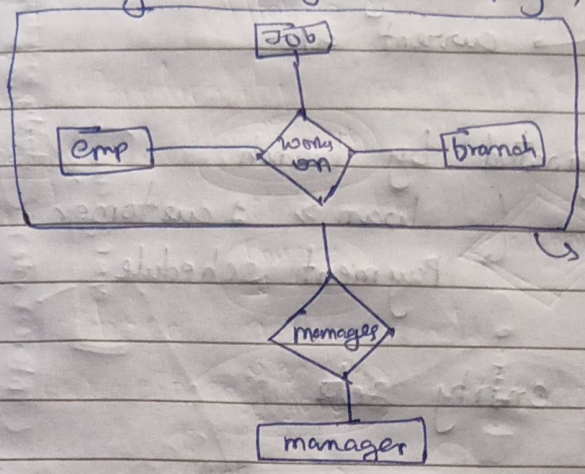
(2) Generalization :- Bottom up approach, reverse of specialization

* Attribute inheritance

Both specialization & Generalization has attribute inheritance → higher level entity sets are inherited by lower level entity sets

* If parent set is involved in a relationship then child entity set will also be involved in relation' participation

Ex a manager manages $employee, job, branch?



Aggregation is a technique which shows relationships among relationship

Abstraction is applied to treat relationships as higher level level entities, we call it abstract entity.

Avoid redundancy by aggregating relationships as an entity set itself.

How to think & formulate an ER diagram

* Steps to make ER diagrams

① Identify Entity sets
② Identify attributes & their types
③ Identify relationships & constraints
   ↳ Mapping
   ↳ Participation

* ER model of Banking System

① Requirement gathering for DB
   → (i) Banking system → Branches (name)
   (ii) Bank → Customers
   (iii) Customers → account & take loan
   (iv) Customer associated with some banker

⑥ Accounts → saving a/c
↳ current a/c

⑦ Loan originated by branch
↳ loan → 1 customer,
Payment schedule

step① Identify entity sets

① Branch
② Customer
③ Employee
④ saving a/c (gen) → account
⑤ Current a/c
⑥ Loan
⑦ Payment (loan) (weak entity)

step② Identify attributes

① Branch :- name, city, assets, liabilities
② Customer :- Customer_id, name, address (composite), contact no. (multivalued), DOB, age (derived)
③ Employee :- emp_id, name, ~~custom~~ contact no., dependent name (multivalued), year of services (derived), start-date
④ Saving account :- acc_no, balance, interest rate, daily withdrawal limit
⑤ Current a/c → acc_no, balance, per transaction charges, overdraft-amt

* Generalize Entity: "Account" → acc_no, balance
⑥ Loan → loan number, amount
⑦ Weak Entity Payment → Payment no., date, amount

PAGE NO.
DATE / /

③ Identify relationships & constraints

① customer borrow Loan
M : : = N

② Loan originated by branch
N = : 1

③ Loan Loan-payment = Payment
1 : N

④ Customer deposit account
M : N

⑤ Customer banker Employee
N : 1

⑥ Employee. managed Employee

H.W

① Online food delivery system
② University

① customer ② order ③ delivery man
④ restaurant ⑤ menu ⑥

① Customer : customer_id, address, name, ph. no

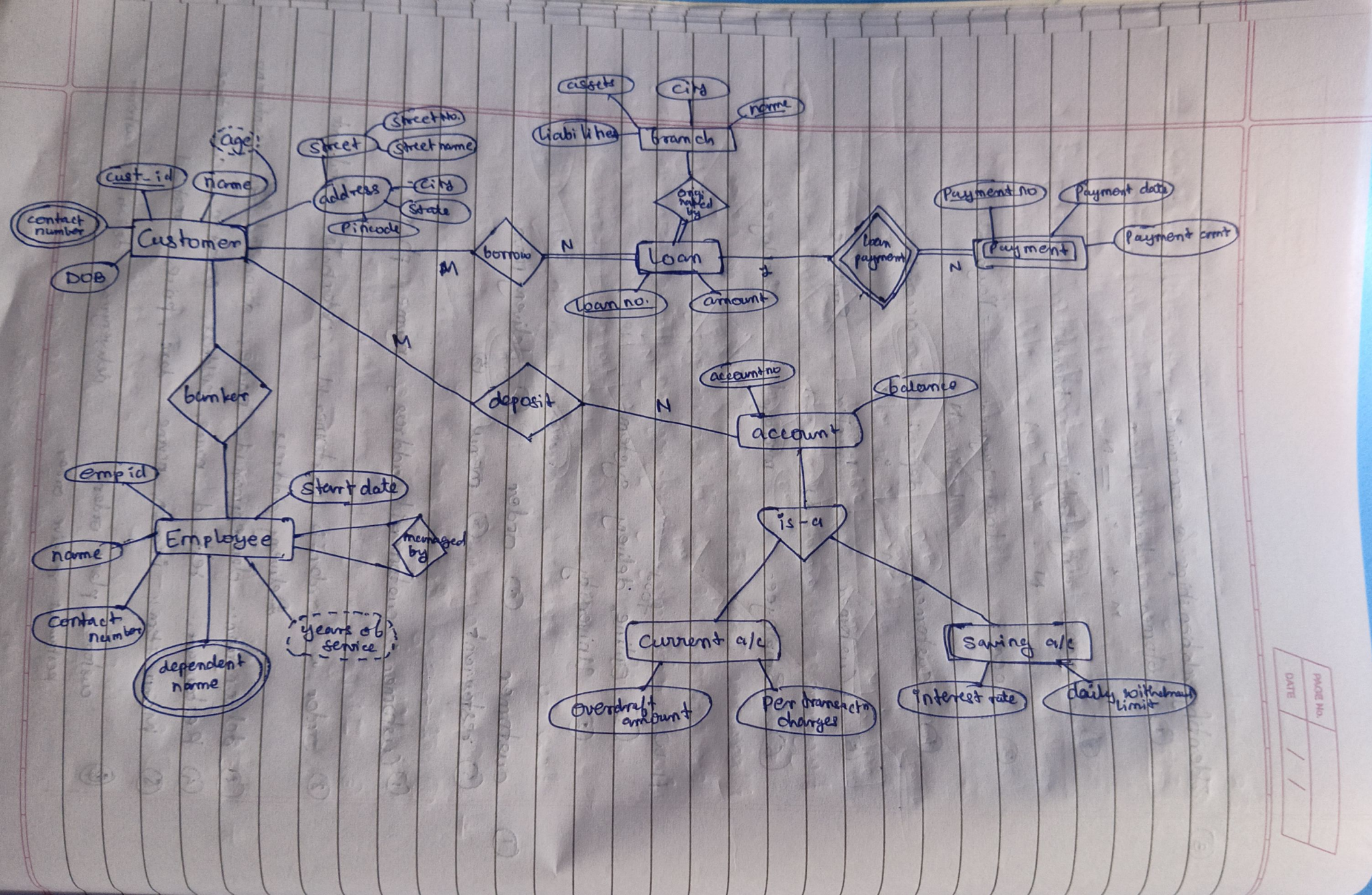
② order : orderID, customer id, orderDate, totalamt, Delivery address

③ deliveryman : deliveryman_id, name, associated area, contact no

④ Restaurant : rest_id, name, address, contact no., Cuisine type

⑤ Menu Item : itemID, name, desc, price, rest_id

⑥ customer place order
Restaurant receive order
deliveryman deliver to customer


assets
City
name
Liabilities
branch
Street no.
Street
Street name
age
cust_id
name
address
City
State
Pincode
contact number
Customer
DOB
managed by
borrow
N
M
Loan
Loan no.
amount
Loan payment
Payment no
Payment date
Payment amnt
Payment
banker
deposit
M
N
account no
balance
account
empid
start date
Employee
name
contact number
dependent name
years of service
managed by
is-a
Current a/c
Saving a/c
Overdraft amount
per transaction charges
Interest rate
daily withdrawal limit

Facebook ER model

(1) Requirement

(1) Features & use case

(1) profile → user_profiles → friends

(2) user can post

(3) post → contains → text content, images, videos

(4) Post → like, comment

(2) Identify entity set

(1) user_profile (2) user_post (3) post_comment

(4) post_like

~~Relⁿ & constraint~~

(2) Attributes

(1) user_profile :- Name, username, email, password, contact no, DOB, age

Name → composite

age → derived

email → multivalued

contact no → multivalued

(2) user_post :- post_id, text content, images, video, created_at, modified_at

images → multivalued

(3) post_comment → comment_id, text content, timestamp

(4) post_like → postlike_id, timestamp

(3) Relation

user_profile — makes — post

1 : N

user_profile — friendship — user_profile

M : N

user_profile — can — post_like

1 : N

User-profile — comments — post comment

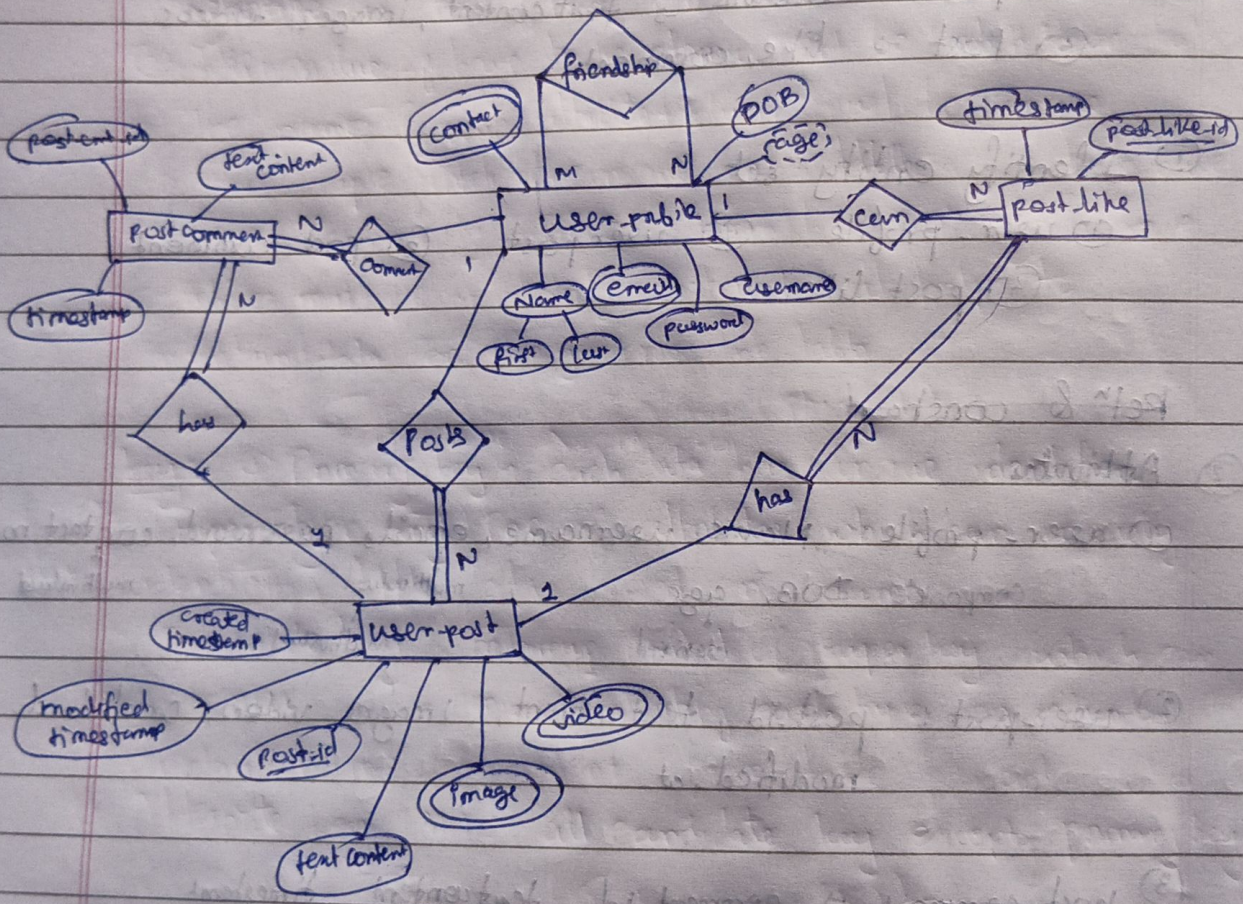
1 : N

user-post — has — post-comment

1 : N

user-post — has — post-like

1 : N



PAGE NO.
DATE / /

- derived attributes → not considered in table

Conversion of Facebook ER diagram

- user_profile: (username, ~~firstName~~, lastName, password, DOB)
- user_email: (username {F.K}, email)
- user_contact: (username {F.K}, contact_number)
- friendship: (profile_req {F.K}, profile_accept {F.K}) → compound key
- post_like: (post_like_id, timestamp, post_id {F.K}, username {F.K})
- ~~post~~ user_post: (post_id, created_at, updated_at, text_content, username {F.K})
- user_post_image: (post_id {F.K}, image_url)
- user_post_video: (post_id {F.K}, video_url)
- post_comment: (post_comment_id, text_content, timestamp, post_id {F.K}, username {F.K})

Note :- 1 cell → 1 value (atomicity)

Q. R_1 & R_2 are two relations containing N1 & N2 tuples respectively, s.t $N2 > N1 > 0$, determine maximum & minimum tuples produced by each of the foll" operation

~~also state~~ ~~N1 > N2~~

a) $R_1 \cup R_2$ — min → N_2, Max → $N_1 + N_2$

(b) $R_1 \cap R_2$ — min → 0, Max → N_1

(c) $R_1 - R_2$ — min → 0, Max → N_1

(d) $R_1 \times R_2$ — min/max → $N_1 \times N_2$

(e) $\sigma_{(a=10)} R_1$ — min → 0, max → N_1

(f) $\Pi_b (R_2)$ — min → 1, max → N_2

Q. How many tuples will be returned by the foll" relational Algebra query

$\Pi_{booktitle}(book)$

Integrity constraints

CRUD operations must be done with some integrity policy so that DB is always consistent & not accidentally corrupted.

1) Domain Constraint :- (i) specify data types of every attribute
(ii) can apply conditions to values

2) Entity constraints :- Every relation should have PK & PK != NULL

3) Referential constraints :- specified between two relations & help maintain consistency among two tuples of relation
→ Insert constraint :- value can't be inserted in child table if the value is not lying in parent table.
→ delete constraint → value can't be deleted from parent if the value is lying in child table

* On delete cascading :- we can delete entry from parent table if the value is lying in the child table w/o violating constraints, by passing on delete cascading.

Q. Can a foreign key be NULL?
On delete NULL :- if record from parent is deleted then the FK in child table in corresponding tuple will be set NULL.

Key constraints :-

1) NOT NULL :- The attribute in the table cannot be a NULL value

2) Unique :- The attribute's value will always be unique from other's in the column

3) DEFAULT :- set a default value of the col^{m}

4) CHECK :- limit value range

5) PRIMARY KEY Constraint :- uniquely identify each tuple
P.k ≠ NUL

In a relation there ~~is~~ can only be one P.k.

6) FOREIGN KEY :- keep relation b/w 2 tables

Transformation from EK model to Relational model

a) strong entity → table/ relation
↳ single valued attributes → col^{m}/attribute

b) weak entity → table/relation
→ has PK + FK & PK+FK → P.k for weak relation

c) composite attribute Eg address
city ✓ street ↓
( ) ...
we'll make seperate attributes for every component

d) Multivalued attribute → make a new relation
Eg dependent name, F.K

emp-id	dependent name

Relational model

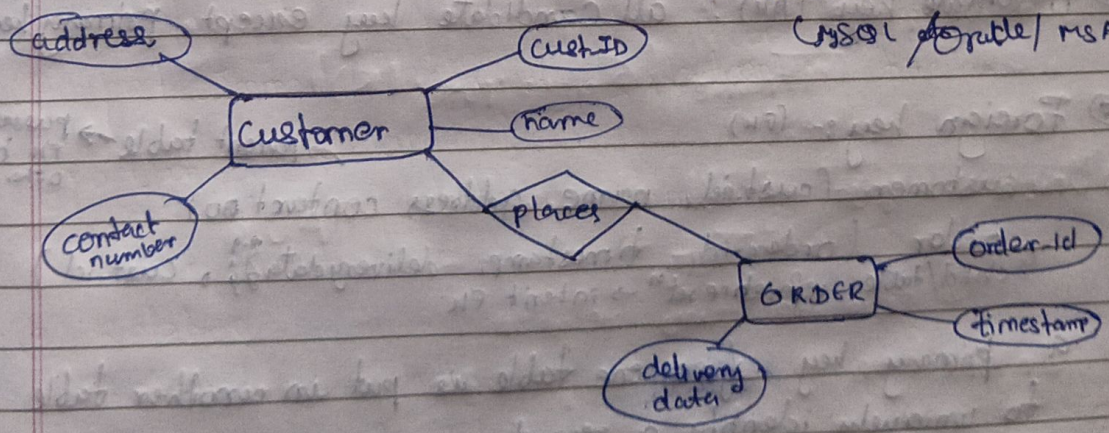
Tables → Customer → - customer_id
↳ Relation itself - Name
- address
- Contact

Table → Relation →	Cust_ID	Name	address	contact
①	1	Lakshay	——	888...
②	2	Raj	——	986...

Degree of Table → no. of attributes / column

Cardinality → Total no of tuple / rows / records

DB design →
① ER Model → ER diagram
↕
Relational model → RDBMS implements the model (MySQL / Oracle / MS Access)



① Customer (cust_ID, name, caddress, contactno)
② ORDER (order_ID, timestamp, delivery_date)

key → foreign key
↳ used to relate two tables

* Properties of table/Relation in Relational Model
① name of table is distinct from among all other relations
② The values of have to be atomic. Can't be broken down further
③ The name of each attribute/column must be unique
④ Each tuple must be unique in a table
⑤ The sequence of row & column has no significance
⑥ Tables must follow integrity constraints, it helps to maintain data consistency across the table

keys ① Primary key → each data point has unique identifier.
② Super key → any Pnc of attributes which can uniquely identify a tuple. (can be NULL)
③ Candidate key (CK) :- minimum subsets of super key, which can uniquely identify each tuple. no redundant attribute (cannot be NULL)
④ Alternate key (AK) :- all candidate key except primary key
⑤ Foreign key :- (FK)
→ parent table → passes FK to other table
customer (cust_id, name, address, contact no)
order (order_id, timestamp, delivery date, cust_id)
child/table reference rel^n → inherit FK
a primary key from one table is put in another table to uniquely identify a record

⑥ Composite key :- primary key formed using at least 2 attributes

⑦ Compound key :- Primary key is formed using 2 foreign key

⑧ Surrogate key :- a synthetic primary key, generated automatically by DB, using usually an int value may also be used as primary key

Integrity Constraints

» CRUD operations must be done with some integrity policy. so that DB is always consistent. they are introduced so » that we do not accidently corrupt the DB

Relational Alegebra

The~~o~~retical operations which we can perform on relations. they also have closure property.

1) Selection operation (σ) (unary operator)

Objective is to filter out table rows we based on condition

$\sigma_{city='PUNE'}(student)$

↳ filter the rows, not deleting, which have city = Pune

combine multiple condition

$\sigma_{city='PUNE' \wedge CGPA>8.0}(student)$

2) Projection (Π)

To filter data based on column

$\Pi_{col1}(R)$

display name of student from table 'student' who live in pune

$\Pi_{name}(\sigma_{city='PUNE'}(R))$

1) Union : ~~list of all cities where~~ Π_{city} $A \cup B$

↳ combine two relation

2) Intersection $A \cap B$

3) difference $A - B$

4) Cartesian Product $(A \times B)$

Join :- To combine two table

① Theta join (θ) :- Combines tuples from two relations where a condition is satisfied

Student ⋈ Marks
Student.Mis = Marks.RollNo (Permits =, >, <, !=, ...)

2) Equi Join :- special case of θ join where condition is only equality (=)

3) Natural Join :- no need of writing condition join on all attribute with same name
Ex Student.Mis & Marks.Mis

Student ⋈ Marks
The common column in both tables will appear only once after natural join

③ Outer join :- To include missing data.

Emp

Emp Name	Street	City
Ram	A	AA
Shyam	B	BB
Rohan	C	CC
Jagat	D	DD

Works

Emp Name	past	Salary
Ram	X	EF
Shyam	Y	2B
Shivansh	Z	9X
Jagat	N	YN

Emp ⋈ Works

Emp Nam	Street	City	past	Salary
Ram	A	AA	X	EF
Shyam	B	BB	Y	2B
Jagat	D	DD	N	YN

Left Join (⟕)

Join missing data from left Table

Emp	Street	City	post	salary
Ram	A	AA	X	EE
Shyam	B	BB	Y	LB
Jagat	D	DD	N	YN
Rohan	C	CC	NULL	NULL

Right Join (⟖) (join missing Data from right table)

Full Outer (⟗)

(join missing Data from both side)

Division operator

~~IDNo~~

student

IDNo	Sports
101	chess
102	Hockey
102	chess
105	Chess

sports_detail

Sports
chess
hockey

student ÷ sports_detail

⇒

IDNo
102